Parameterized notebooks

It is often useful to render the same notebook with different inputs. Calepin document parameters let Typst source code or command-line flags pass values to execution engines. Code chunks can then read those values through a `params` object.

For the `r` and `python` engines, Calepin creates the binding automatically: R receives a named `list` read as `params$Species`, and Python receives a `dict` read as `params["Species"]`.

This complete document filters the built-in R `iris` data to one species and a minimum petal length, then uses another parameter to choose the color palette.

````
#import "@preview/calepin:0.0.1" as calepin

#calepin.setup(
  params: (
    Species: "versicolor",
    min_petal_length: 4.5,
    palette: "viridis",
  ),
)

The selected species is #calepin.inline("r")[`cat(params$Species)`].

```r
#| fig-caption: Iris rows selected by document parameters
filtered <- subset(
  iris,
  Species == params$Species & Petal.Length >= params$min_petal_length
)

colors <- hcl.colors(3, palette = params$palette)

plot(
  Sepal.Length ~ Petal.Length,
  data = filtered,
  pch = 19,
  col = colors[2],
  xlab = "Petal length",
  ylab = "Sepal length",
  main = paste(params$Species, "with Petal.Length >=", params$min_petal_length)
)
```
````

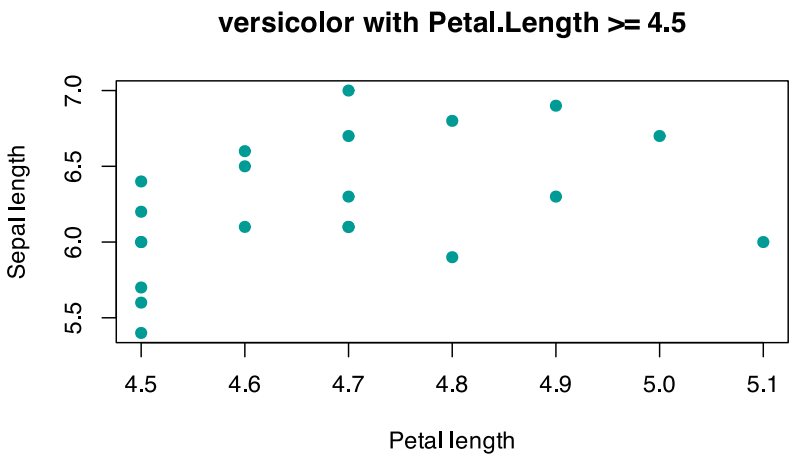
The selected species is versicolor.

```r
filtered <- subset(
  iris,
  Species == params$Species & Petal.Length >= params$min_petal_length
)

colors <- hcl.colors(3, palette = params$palette)

plot(
  Sepal.Length ~ Petal.Length,
  data = filtered,
  pch = 19,
  col = colors[2],
  xlab = "Petal length",
  ylab = "Sepal length",
```

```
  main = paste(params$Species, "with Petal.Length >=", params$min_petal_length)
)
```



Because parameters are defined in `#calepin.setup`, command-line values can override them at render time with `-P key=value` (repeatable). This renders the same source with different inputs without editing the notebook:

```
calepin compile iris.typ -P Species=setosa -P min_petal_length=1.5 -P palette=magma
```

Command-line values are typed the same way as Quarto-style header values, so `1.5` is a number, `true` is a boolean, and `setosa` is a string.

Parameter values may be `none`, a boolean, an integer, a float, a string, an array, or a dictionary, nested freely. Other Typst values such as content, functions, lengths, colors, and dates cannot be passed directly; supply them as strings or numbers instead. Unsupported values fail the build with a message naming the offending parameter.

Parameters are also written to `.calepin/<document>/params.json`. Engines reached through a Jupyter kernel, including `julia` and any other kernel, do not yet receive an automatic `params` binding. Read that JSON file yourself using the `CALEPIN_PARAMS_PATH` environment variable that Calepin sets in the kernel. Parameters are not secret: treat them as build inputs that are written to disk.